

Entrega Guia 1: Gestión de tareas

April 23, 2026

1 Ejercicio 1

1.1 Implementación

Las tareas implementadas son todas de esta pinta, con la diferencia del delay utilizado para setear la variable Delay_ms y los leds utilizados son uno distinto para cada tarea.

```
void vTareaParpadeo200(void *pvParameters){  
    const TickType_t Delay_ms = pdMS_TO_TICKS( 200 );  
    GPIO_TypeDef* port = LD4_GPIO_Port;  
    uint16_t pin = LD4_Pin;  
    while (1){  
        HAL_GPIO_TogglePin( port , pin );  
        HAL_Delay( Delay_ms );  
    }  
}
```

Y se crean las tareas de esta manera

```
xTaskCreate(  
    vTareaParpadeo200 ,  
    "Blink200",  
    configMINIMAL_STACK_SIZE ,  
    NULL ,  
    tskIDLE_PRIORITY ,  
    NULL  
);  
... // 400ms, 600ms  
  
xTaskCreate(  
    vTareaParpadeo800 ,  
    "Blink200",  
    configMINIMAL_STACK_SIZE ,  
    NULL ,  
    tskIDLE_PRIORITY ,  
    NULL  
);
```

1.2 Conclusiones

Todas las tareas se ejecutan. Esto es obvio pues cada tarea tiene asignado un led distinto y estos están funcionando a periodos diferentes.

Pareciera que hay concurrencia pero dado que el procesador es de un solo nucleo, sabemos que es una ilusión provocada por el scheduling de FREERTOS. Mas claramente comprobamos en clase usando el osciloscopio, hay una medible diferencia entre el inicio de de una tarea a pesar de que esto es aparentemente en el mismo instante.

Los tiempos de conmutación no serán precisos. Pues dado que las esperas son no-bloqueantes. El scheduler tendrá que intervenir para repartir la cpu y en dichos cambios de contexto se introduce jitter.

2 Ejercicio 2

2.1 Implementación

La única tarea implementada es

```
void vTareaParpadeo(void *pvParameters){
    // Recibir parametros
    struct ParpadeoParameters parameters = \
        *(struct ParpadeoParameters *) pvParameters;

    const TickType_t xDelaysms = pdMS_TO_TICKS( parameters.ms );
    while (1){
        HAL_GPIO_TogglePin(parameters.Port, parameters.Pin);
        vTaskDelay( xDelaysms );
    }
}
```

Y se inician en main, las tareas:

```
// Crear las configuraciones para cada tarea
static struct ParpadeoParameters task200Parameters = {
    .Pin = LD4_Pin,
    .Port = LD4_GPIO_Port,
    .ms = 200
};

// ... 400ms, 600ms, y leds 5 y 3
static struct ParpadeoParameters task800Parameters = {
    .Pin = LD6_Pin,
    .Port = LD6_GPIO_Port,
    .ms = 800
};

xTaskCreate(
    vTareaParpadeo,
    "Blink200",
    configMINIMAL_STACK_SIZE,
    &task200Parameters,
    tskIDLE_PRIORITY,
    NULL
);

// ... 400ms, 600ms
xTaskCreate(
    vTareaParpadeo,
    "Blink800",
    configMINIMAL_STACK_SIZE,
    &task800Parameters,
    tskIDLE_PRIORITY,
    NULL
);
```

2.2 Conclusiones

En el Desafío 2 las tareas utilizan `vTaskDelay`, lo que introduce estados de bloqueo (Blocked). A diferencia del Desafío 1, donde las tareas permanecían activas usando busy-wait, ahora ceden el CPU voluntariamente. Esto permite al scheduler gestionar mejor la ejecución, reduce la competencia por CPU y disminuye el jitter. Como resultado, los períodos de conmutación son más estables, aunque siguen limitados por la resolución del tick del sistema.

3 Ejercicio 3

3.1 Implementación

```
void vHandlePush(void *pvParameters) {
    while(1){
        if ( HAL_GPIO_ReadPin(B1_GPIO_Port , B1_Pin) ==
GPIO_PIN_SET ){
            HAL_GPIO_WritePin(LD6_GPIO_Port , LD6_Pin ,
GPIO_PIN_SET);
        }
        else if ( HAL_GPIO_ReadPin(B1_GPIO_Port , B1_Pin) ==
GPIO_PIN_RESET ){
            HAL_GPIO_WritePin(LD6_GPIO_Port , LD6_Pin ,
GPIO_PIN_RESET);
        }
        vTaskDelay(pdMS_TO_TICKS(10)); // Bloquearse 10ms
    }
}

void vTareaParpadeo(void *pvParameters){
    // Recibir parametros
    struct ParpadeoParameters parameters = *(struct
ParpadeoParameters *) pvParameters;
    while (1){
        HAL_GPIO_TogglePin(parameters.Port , parameters.Pin);
        HAL_Delay( 500 );
    }
}
```

`vHandlePush` hará polling cada 10ms para verificar el estado del botón y aplicar el estado correspondiente al LED. La otra tarea se reutilizó de ejercicios pero ahora es no-bloqueante por usar `HAL_Delay` como servicio de espera.

Se crean las tareas de esta manera

```
static struct ParpadeoParameters LedBloqueante500ms = {
    .Pin = LD4_Pin,
    .Port = LD4_GPIO_Port,
    .ms = 500
};

xTaskCreate(
    vTareaParpadeo,
    "LedBloqueante500ms",
    configMINIMAL_STACK_SIZE,
    &LedBloqueante500ms,
    2,
    NULL
);
xTaskCreate(
    vHandlePush,
    "PollingBoton",
    configMINIMAL_STACK_SIZE,
    NULL,
    3,
    NULL
);
```

3.2 Conclusiones

Al usar tiempos de `vTaskDelay` mas altos, el botón responde muy tarde al pulsar y soltar. Y es posible hallar ventanas donde se puede presionar y soltar sin respuesta de ningún tipo. Un `vTaskDelay` de 10ms satisface la consigna, pues asegura que la tarea no estaría más de 10ms bloqueada y sabemos que una vez se ponga `READY`, el scheduler decide ejecutarla. Esto no se asegura con total precision, pues se introduce un jitter por el cambio de contexto, dado que la otra tarea siempre está ejecutando el resto del tiempo.

4 Ejercicio 4

4.1 Implementación

```
void vTareaParpadeoA(void *pvParameters){
    struct ParpadeoParameters parameters = *(struct
ParpadeoParameters *) pvParameters;
    while (1){
        HAL_GPIO_TogglePin(parameters.Port, parameters.Pin);
        HAL_Delay( 100 ); // Trabajo simulado
        vTaskDelay(pdMS_TO_TICKS(parameters.ms)); // Handlear
cada 10ms
    }
}

void vTareaParpadeoB(void *pvParameters){
    struct ParpadeoParameters parameters = *(struct
ParpadeoParameters *) pvParameters;
    TickType_t xLastWakeTime;
    xLastWakeTime = xTaskGetTickCount();
    const TickType_t xFrequency = pdMS_TO_TICKS(parameters.ms);
    while (1){
        HAL_GPIO_TogglePin(parameters.Port, parameters.Pin);
        HAL_Delay( 100 ); // Trabajo simulado
        vTaskDelayUntil(&xLastWakeTime, xFrequency);
    }
}
```

Notar que la TareaB utiliza vTaskDelayUntil en vez de vTaskDelay como en TareaA.

Las tareas se crean con la misma prioridad, y usando LED's diferentes. Notar que se setea el campo ms de forma diferente

```
static struct ParpadeoParameters paramsTareaA = {
    .Pin = LD4_Pin,
    .Port = LD4_GPIO_Port,
    .ms = 400
};
static struct ParpadeoParameters paramsTareaB = {
    .Pin = LD5_Pin,
    .Port = LD5_GPIO_Port,
    .ms = 500
};

xTaskCreate(
    vTareaParpadeoA,
    "LedBloqueante-Delay500ms",
    configMINIMAL_STACK_SIZE,
    &paramsTareaA,
    0,
    NULL
);
xTaskCreate(
    vTareaParpadeoB,
    "LedBloqueante-DelayUntil500ms",
    configMINIMAL_STACK_SIZE,
    &paramsTareaB,
    0,
    NULL
);
```

4.2 Conclusiones

El tiempo utilizado en la TareaA es de 400ms, pues para mantener una periodicidad de 500ms con un trabajo de 100ms necesitamos compensar el retardo agregado por el trabajo. En el caso de TareaB, vTaskDelayUntil es capaz de compensar el retardo agregado por el trabajo de 100ms pues considera el tiempo absoluto desde que inicio la tarea.

Si ambos argumentos son iguales de 500ms, se observa un desfase muy exagerado en la conmutación de los LED's. Dado que la TareaA termina el trabajo de 100ms y es entonces que comienza a esperar 500ms para desbloquearse. En cambio la TareaB considera la diferencia.